

Public Home Page & Free-to-Use Chat

Removing the forced-registration wall: the home page and its chatbot are now open to anonymous visitors, while every feature still requires an account to actually use.

Prepared for: Mohammed (Mizan.ai)

Scope: Auth flow change (frontend + backend) and redeployment to GCP

Purpose: Reference for personal understanding, technical interviews, and live demos

1. What changed and why

Previously, every visitor to `mizanai.org` was redirected straight to `login.html` before seeing anything at all — the home page itself required a valid login. The new behavior:

	Before	After
Home page	Redirects to login immediately	Open to anyone, no account needed
Home page chatbot	Not reachable without logging in first	Usable by anonymous visitors
Feature cards (VAT guidance, translator, calculator, comparator, converter)	Required login	Still require login — clicking one redirects to <code>login.html</code>

Why this is a two-layer change, not just a frontend tweak: the frontend redirect was only half the gate. The chat API itself (`POST /api/chat` in `app/main.py`) required a valid JWT on the *backend* too, independent of anything the browser did. Removing only the frontend redirect would have let a guest load the page, but their very first chat message would still have been rejected by the server with `401 Unauthorized` . Both sides needed to change together.

2. Backend change: an optional-auth path for chat

Mizan.ai's JWT auth (`app/auth.py`) already had two dependency functions used across the API:

Dependency	Behavior
<code>get_current_user_id</code>	Requires a valid token; accepts a pending-password-reset token too (used only by the reset-password endpoint itself)
<code>get_current_user_id_full_access</code>	Requires a valid token; rejects a pending-password-reset token — used by every normal protected business endpoint

Neither of these could return "no user, but let the request through anyway" — both always either resolve a real user or raise `401` . A third dependency was added for exactly that case:

```
def get_current_user_id_optional(
    credentials: Optional[HTTPAuthorizationCredentials] = Depends(_bearer_scheme),
) -> Optional[str]:
    """Never rejects the request. Returns the user_id for a valid, fully-usable
    token, and None for anything else: no token, an invalid/expired one, or a
    pending-password-reset one."""
    if credentials is None:
        return None
    try:
        payload = _decode_token(credentials.credentials)
    except ValueError:
        return None
    if payload.get("must_reset_password"):
        return None
    return payload["sub"]
```

`/api/chat` was switched from `get_current_user_id_full_access` to this new dependency, with its `user_id` parameter changed to `Optional[str]`:

```
@app.post("/api/chat", response_model=ChatResponse, tags=["D - chat"])
def chat(req: ChatRequest, user_id: Optional[str] = Depends(get_current_user_id_optional)):
    ...
```

Why this was safe to do without touching the chat engine itself: the LangGraph chatbot's `generate_reply()` function already accepted `user_id: Optional[str] = None` as its signature, before this change — it was already designed to tolerate a missing user identity. The only thing `user_id` actually does inside the chat graph is key a per-user profile-memory lookup ("what does Mizan.ai already know about this person from earlier conversations?"). For a guest, that step is simply skipped — they get a normal, correct reply, just without personalization carried over from a past session. Nothing about the chat logic itself needed to change.

3. Frontend change: dropping the gate on the home page

`frontend/app.html` previously started its script block with:

```
mzRequireAuth();
```

which (per `common.js`) immediately redirects to `login.html` if there's no saved token in the browser. That call was removed entirely from the home page.

The nav bar had to change too

With the page open to guests, the top-right nav link — previously a hardcoded "Log out" — no longer made sense for a visitor who was never logged in. It's now built dynamically based on whether a token exists:

```
const authLink = document.getElementById("auth-nav-link");
if (mzGetToken()) {
  authLink.innerHTML = '<a href="#" id="logout-link">Log out</a>';
  document.getElementById("logout-link").addEventListener("click", (e) => { e.preventDefault();
mzLogout(); });
} else {
  authLink.innerHTML = '<a href="login.html">Log in</a> <a href="register.html">Register</a>';
}
```

The chat box itself needed no change

It already called the shared `mzFetch()` helper, which attaches an `Authorization` header *only if* a token exists in `localStorage` — it never required one. So a logged-in visitor's messages are still sent with their token (and get personalized replies from the backend change in Part 2); a guest's are sent with no token at all, which the new optional-auth dependency now accepts instead of rejecting.

4. Why the 5 feature cards needed zero changes

Before touching anything, each destination page was checked:

Card	Destination page	Already calls <code>mzRequireAuth()</code> ?
VAT & ZATCA Guidance	<code>know-vat-zatca.html</code>	Yes
Document Translator	<code>translator.html</code>	Yes
VAT Compliance Center	<code>calculator.html</code>	Yes
Document Reconciliation	<code>comparator.html</code>	Yes
Convert & Edit Files	<code>converter.html</code>	Yes

Result: every one of these pages was already independently gated. Clicking any feature card from the now-public home page still lands the visitor on a page that immediately redirects to `login.html` — exactly the "ask them to register to continue" behavior that was asked for, with no code changes needed on that side at all.

5. A deliberate gap, left open on purpose

No rate limiting exists anywhere in this app today. The `remaining_daily_quota: 29` field returned by `/api/chat` is a hardcoded placeholder, not an enforced limit — this was true before this change too. Previously, registration was at least a weak speed bump against unlimited free LLM usage; making chat anonymous removes even that. This was a conscious decision, not an oversight: LLM usage will be watched manually against the provider's usage dashboard for now, with IP-based rate limiting to be added later only if that actually becomes necessary.

6. Redeploying the change

Two images needed rebuilding and redeploying — `chatbot` (the backend change) and `frontend` (the home page change) — in both places the stack runs: the local `docker-compose` stack, and the GCP k3s cluster behind `mizanai.org`.

Local verification first

```
docker build -f services/chatbot/Dockerfile -t mizanai-chatbot .
docker compose up -d --no-deps chatbot

curl -X POST http://localhost:8000/api/chat -H "Content-Type: application/json" -d
'{"message":"hello"}'
# -> 200, real reply, no Authorization header sent at all
```

Why test locally before touching GCP: confirms the actual behavior change works correctly — a real anonymous request reaching the real chat engine and getting a real reply — before spending time on a full rebuild-push-restart cycle against the production cluster.

Building and pushing to Artifact Registry

```
docker build -f services/chatbot/Dockerfile \
  -t us-central1-docker.pkg.dev/mizan-ai-kalimi03/mizan-images/chatbot:latest .
docker build -f frontend/Dockerfile \
  -t us-central1-docker.pkg.dev/mizan-ai-kalimi03/mizan-images/frontend:latest frontend/

docker push us-central1-docker.pkg.dev/mizan-ai-kalimi03/mizan-images/chatbot:latest
docker push us-central1-docker.pkg.dev/mizan-ai-kalimi03/mizan-images/frontend:latest
```

Restarting both deployments on the cluster

```
gcloud compute ssh k3s-vm --zone us-central1-a --command \
  "sudo kubectl rollout restart deployment/chatbot deployment/frontend -n mizan && \
  sudo kubectl rollout status deployment/chatbot -n mizan --timeout=90s && \
  sudo kubectl rollout status deployment/frontend -n mizan --timeout=60s"
```

Why a rollout restart, not just re-pushing the image: both Deployments use the same `:latest` tag every time, and a running pod doesn't automatically notice that the tag now points at different image content. `kubectl rollout restart` forces Kubernetes to recreate the pod, which — combined with `imagePullPolicy: Always` already set on both Deployments — makes it re-pull the image fresh instead of reusing what's cached on the node.

Final verification, through the real domain

```
curl https://mizanai.org/
# -> 200, home page loads with no auth

curl -X POST https://mizanai.org/api/chat -H "Content-Type: application/json" -d
'{"message":"hello"}'
# -> 200, real reply, no Authorization header sent at all
```

Confirmed working end-to-end: the public home page loads for anonymous visitors, the chatbot answers without requiring login, and every feature card still redirects to the login/register flow when clicked — on both the local stack and the live GCP deployment.

```
Anonymous visitor -> https://mizanai.org/
|
v
Home page loads directly          (mzRequireAuth() removed from app.html)
|
|-- types a chat message --> POST /api/chat, no Authorization header
|
|                               |
|                               v
|                               get_current_user_id_optional() -> None
|                               |
|                               v
|                               generate_reply(..., user_id=None) -> normal reply, no personalization
|
'-- clicks a feature card --> e.g. calculator.html
|
|                               v
|                               mzRequireAuth() (already existed) -> no token -> redirect to login.html
```